

Prueba Técnica: Associate Data Engineer:

Brenda Nicole Henríquez Amaya

Problema de negocio

El objetivo de este proyecto es diseñar e implementar un proceso de ingeniería de datos (ETL) que resuelva una necesidad operativa y analítica dividida en tres fases consecutivas:

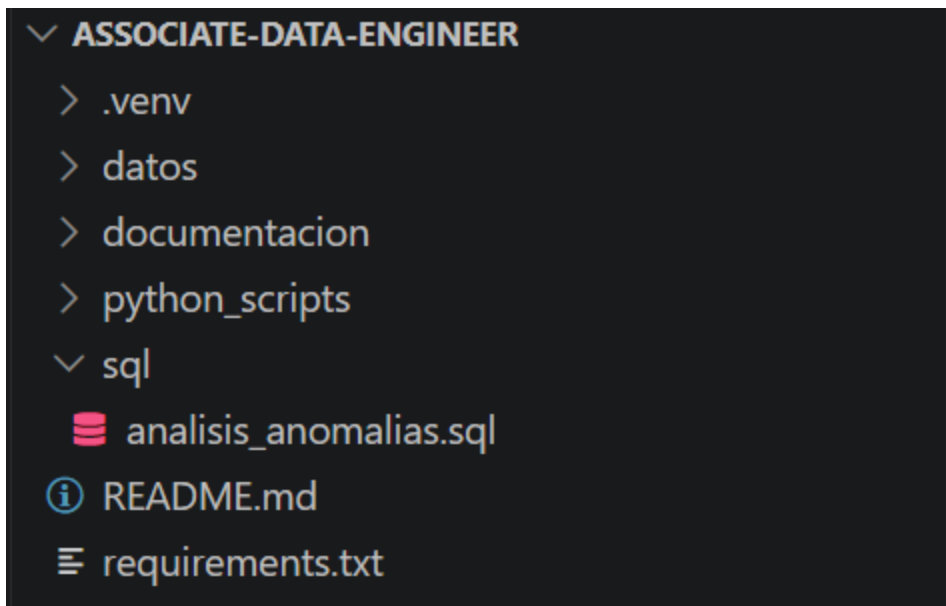
Procesamiento y Carga (Python): Leer un conjunto de datos crudos denominado `transacciones_diarias.csv`, limpiarlo bajo reglas de calidad específicas (manejo de duplicados e imputación de valores nulos) y estructurarlo bajo un modelo relacional de Esquema Snowflake (Esquema de Copo de Nieve). Finalmente, los datos procesados deben cargarse de forma automatizada en una base de datos en la nube (Supabase).

Análisis de Anomalías (SQL): Desarrollar una consulta analítica que explote el modelo relacional cargado. El propósito es identificar patrones inusuales o posibles fraudes financieros, específicamente detectando aquellos usuarios cuyos montos de gasto aprobados se hayan incrementado de forma abrupta en comparación con su transacción inmediata anterior.

Propuesta de Orquestación (Apache Airflow): Diseñar un script conceptual de automatización que programe la ejecución de todo el flujo de trabajo para que corra diariamente a las 11:30 PM, garantizando que el análisis analítico solo se ejecute si la carga de datos previa concluyó con éxito.

Estructura del Proyecto

Para mantener un entorno de desarrollo limpio, el proyecto lo he organizado bajo la siguiente estructura de archivos y carpetas visibles en el espacio de trabajo:



Descripción

- **Aislamiento de Datos (datos/):** El archivo fuente `transacciones_diarias.csv` se extrae de la raíz y se confina a su propio directorio.
- **Scripts y Orquestación (python_scripts/):** Centraliza la lógica de programación en Python. Al agrupar el archivo del DAG junto con el script de transformación, se simplifica el control de versiones y se facilita a Apache Airflow la lectura del comando `python python_scripts/script.py` de forma limpia desde la raíz.
- **Módulo Analítico (sql/):** Espacio dedicado exclusivamente a los scripts de explotación de datos, donde se almacena la información de negocio para auditoría de anomalías financieras.
- **Entorno Virtual y Dependencias (.venv/ y requirements.txt):** Aseguran que cualquier usuario o servidor de Airflow pueda replicar exactamente las librerías necesarias (`pandas` , `psycopg2-binary`) sin alterar el sistema operativo global.

Entregable 1: Script de Limpieza y Carga (script.py)

Este archivo contiene el desarrollo en Python encargado de la Fase de Extracción, Transformación y Carga (ETL). Su objetivo principal es recibir el archivo plano `transacciones_diarias.csv` desde la carpeta de almacenamiento de datos, aplicar transformaciones basadas en reglas de calidad y de negocio, estructurar la información bajo un modelo relacional de tipo **Esquema Snowflake** y cargar el resultado de forma directa y segura en la instancia de base de datos en la nube (Supabase) utilizando conectores nativos optimizados.

1.1 Dependencias y Conexión

El inicio del script importa las librerías necesarias y define la ruta de acceso al motor de base de datos. Se migró el motor de conexión a la librería nativa de PostgreSQL (`psycopg2`) para evitar conflictos de controladores y asimilar la ejecución directamente en el canal de base de datos.

```
import pandas as pd
import psycopg2

# Cadena de conexión estricta hacia la instancia de PostgreSQL en Supabase
# con cifrado SSL
DATABASE_URL =
"postgresql://postgres:basededatos123@db.pz1knfqntvljoduxegk.supabase.co:543
sslmode=require"
```

1.2 Función de Transformación y Normalización (procesar_y_normalizar_snowflake)

Esta función procesa el archivo crudo, imprimiendo el estado actual en consola, y aplica consecutivamente las tres primeras reglas de negocio:

```
def procesar_y_normalizar_snowflake(filepath):
    """
    Lee el archivo CSV, aplica las reglas de calidad y negocio,
    y estructura los datos bajo un modelo relacional Snowflake.
    """

    print(">>> 1. Iniciando lectura del archivo crudo
    'transacciones_diarias.csv'...")
    df = pd.read_csv(filepath)

    # REGLA DE NEGOCIO 1: Limpieza de registros duplicados basados en el
    # identificador único de transacción.
    print(">>> 2. Aplicando Regla 1: Eliminando duplicados por
    id_transaccion...")
    df = df.drop_duplicates(subset=['id_transaccion'], keep='first')

    # REGLA DE NEGOCIO 2: Gestión de valores nulos.
    # Si el monto es ausente y el estado de la transacción fue
    # 'rechazada', se imputa el valor a 0.0.
    print(">>> 3. Aplicando Regla 2: Imputando nulos en transacciones
    rechazadas...")
    condicion_nulo_rechazado = (df['monto_usd'].isna()) &
    (df['estado_transaccion'] == 'rechazada')
    df.loc[condicion_nulo_rechazado, 'monto_usd'] = 0.0

    # REGLA DE NEGOCIO 3: Clasificación de transacciones sospechosas.
    # Se marcan como verdaderas (True) aquellas de carácter internacional
    # con montos que superen los $1500 USD.
    print(">>> 4. Aplicando Regla 3: Clasificando montos inusuales (Monto
    > 1500 e Internacional)...")
    df['es_monto_inusual'] = (df['monto_usd'] > 1500) &
    (df['tipo_comercio'] == 'internacional')
```

Posteriormente, dentro de la misma función, se ejecuta la descomposición relacional para dar forma al Esquema Snowflake, aislando las entidades de negocio para eliminar redundancias operacionales:

```
print(">>> 5. Normalizando estructura para el esquema Snowflake...")
    # 1. Dimensión Clientes: Extrae Los identificadores únicos de cada
    cliente presente en los datos.
    dim_clientes =
df[['id_cliente']].drop_duplicates().reset_index(drop=True)

    # 2. Dimensión Comercios: Aísla el tipo de comercio y genera un código
    numérico secuencial (Llave Primaria).
    dim_comercios =
df[['tipo_comercio']].drop_duplicates().reset_index(drop=True)
    dim_comercios['id_comercio'] = dim_comercios.index + 1

    # 3. Vinculación: Cruzamos el DataFrame original con la dimensión de
    comercios para asociar a cada registro su respectivo ID.
    df_fact = df.merge(dim_comercios, on='tipo_comercio', how='left')

    # 4. Tabla de Hechos (Fact Table): Se seleccionan únicamente las
    llaves foráneas y las métricas calculadas.
    fact_transacciones = df_fact[[
        'id_transaccion',
        'id_cliente',
        'id_comercio',
        'fecha_hora',
        'monto_usd',
        'estado_transaccion',
        'es_monto_inusual'
    ]]

    return dim_clientes, dim_comercios, fact_transacciones
```

1.3 Función de Carga (cargar_a_supabase_snowflake)

Esta función interactúa de forma directa con Supabase usando un cursor nativo. Remueve de manera limpia las tablas previas mediante un borrado en cascada para evitar colisiones y realiza inserciones iterativas directas, garantizando el control absoluto de tipos y restricciones transaccionales:

```
def cargar_a_supabase_snowflake(dim_cli, dim_com, fact_trans):
    """
    Se conecta a Supabase de forma nativa con psycopg2, limpia y carga
    datos
    usando conectores puros para evitar conflictos de SQLAlchemy.
    """
    print(">>> 6. Estableciendo conexión segura nativa con Supabase...")

    try:
```

```

        # Abrimos conexión directa con la base de datos usando el driver
nativo de Postgres
        conn = psycopg2.connect(DATABASE_URL)
        cursor = conn.cursor()

        print(">>> 7. Limpiando tablas previas si existen (Evita
conflictos)...")
        cursor.execute("DROP TABLE IF EXISTS fact_transacciones CASCADE;")
        cursor.execute("DROP TABLE IF EXISTS dim_clientes CASCADE;")
        cursor.execute("DROP TABLE IF EXISTS dim_comercios CASCADE;")

        # Carga transaccional mapeada para evitar errores estructurales de
ORM
        print(">>> 8. Cargando datos a la dimensión 'dim_clientes'...")
        cursor.execute("CREATE TABLE dim_clientes (id_cliente TEXT);")
        for val in dim_cli['id_cliente']:
            cursor.execute("INSERT INTO dim_clientes (id_cliente) VALUES
(%)");", (str(val),))
        cursor.execute("ALTER TABLE dim_clientes ADD PRIMARY KEY
(id_cliente);")

        print(">>> 9. Cargando datos a la dimensión 'dim_comercios'...")
        cursor.execute("CREATE TABLE dim_comercios (tipo_comercio TEXT,
id_comercio INT);")
        for _, row in dim_com.iterrows():
            cursor.execute("INSERT INTO dim_comercios (tipo_comercio,
id_comercio) VALUES (%s, %s);", (str(row['tipo_comercio']),
int(row['id_comercio'])))
        cursor.execute("ALTER TABLE dim_comercios ADD PRIMARY KEY
(id_comercio);")

        print(">>> 10. Cargando datos a la tabla de hechos
'fact_transacciones'...")
        cursor.execute("""
            CREATE TABLE fact_transacciones (
                id_transaccion TEXT, id_cliente TEXT, id_comercio INT,
                fecha_hora TEXT, monto_usd NUMERIC, estado_transaccion
TEXT, es_monto_inusual BOOLEAN
            );
        """)
        for _, row in fact_trans.iterrows():
            cursor.execute("""
                INSERT INTO fact_transacciones (id_transaccion,
id_cliente, id_comercio, fecha_hora, monto_usd, estado_transaccion,
es_monto_inusual)
                VALUES (%s, %s, %s, %s, %s, %s, %s);
            """, (str(row['id_transaccion']), str(row['id_cliente']),
int(row['id_comercio']), str(row['fecha_hora']), float(row['monto_usd']),
str(row['estado_transaccion']), bool(row['es_monto_inusual'])))
        cursor.execute("ALTER TABLE fact_transacciones ADD PRIMARY KEY
(id_transaccion);")

```

```

        # Inyección de DDL para asegurar La Integridad Referencial del
esquema Snowflake
        print(">>> 11. Creando llaves foráneas (Relaciones del Esquema
Snowflake)...")
        cursor.execute("""
            ALTER TABLE fact_transacciones
            ADD CONSTRAINT fk_cliente FOREIGN KEY (id_cliente) REFERENCES
dim_clientes(id_cliente),
            ADD CONSTRAINT fk_comercio FOREIGN KEY (id_comercio)
REFERENCES dim_comercios(id_comercio);
        """)

        # Confirmamos todos los cambios en la BD en un único bloque
atómico y cerramos canales
        conn.commit()
        cursor.close()
        conn.close()

        print("¡ÉXITO EN EL PIPELINE! El modelo Snowflake fue estructurado
y poblado correctamente en Supabase.")
    except Exception as e:
        print(f"❌ Error crítico durante la carga en la base de datos:
{e}")

```

1.4 Bloque de Ejecución Principal (main)

Determina el punto de entrada del script. Llama ordenadamente al proceso completo utilizando la nueva ruta de navegación relativa (../datos/) requerida por la reestructuración modular del repositorio.

```

if __name__ == "__main__":
    # Ruta del archivo origen adaptada a la nueva carpeta modular de
almacenamiento
    archivo_origen = "../datos/transacciones_diarias.csv"

    # Ejecución completa del proceso ETL
    dim_cli, dim_com, fact_trans =
procesar_y_normalizar_snowflake(archivo_origen)
    cargar_a_supabase_snowflake(dim_cli, dim_com, fact_trans)

```

Resultado

```

>>> 1. Iniciando lectura del archivo crudo 'transacciones_diarias.csv'...
>>> 2. Aplicando Regla 1: Eliminando duplicados por id_transaccion...
>>> 3. Aplicando Regla 2: Imputando nulos en transacciones rechazadas...
>>> 4. Aplicando Regla 3: Clasificando montos inusuales (Monto > 1500 e Internacional)...
>>> 5. Normalizando estructura para el esquema Snowflake...
>>> 6. Estableciendo conexión segura nativa con Supabase...
>>> 7. Limpiando tablas previas si existen (Evita conflictos)...
>>> 8. Cargando datos a la dimensión 'dim_clientes'...
>>> 9. Cargando datos a la dimensión 'dim_comercios'...
>>> 10. Cargando datos a la tabla de hechos 'fact_transacciones'...
>>> 11. Creando llaves foráneas (Relaciones del Esquema Snowflake)...
¡ÉXITO EN EL PIPELINE! El modelo Snowflake fue estructurado y poblado correctamente en Supabase.

```

Entregable 2: Consulta Analítica de Anomalías (analisis_anomalias.sql)

Este entregable contiene el desarrollo en lenguaje SQL encargado de la **Fase 2 (Análisis de Anomalías)**. Su propósito es interrogar el modelo relacional estructurado en Supabase para identificar de manera automatizada patrones de comportamiento inusuales o potenciales fraudes financieros, cumpliendo estrictamente con la **Regla de Negocio 4**: detectar transacciones aprobadas cuyo monto actual sea igual o mayor a cinco veces el monto de la transacción inmediatamente anterior del mismo cliente.

Codigo del archivo:

```
-- Identificación de incrementos bruscos de dinero utilizando funciones de
-- ventana sobre el modelo Snowflake
WITH transacciones_validas AS (
    SELECT
        id_transaccion,
        id_cliente,
        fecha_hora,
        monto_usd,
        -- Obtenemos cronológicamente el monto anterior del mismo cliente
        LAG(monto_usd) OVER(PARTITION BY id_cliente ORDER BY fecha_hora)
    AS monto_anterior
    FROM fact_transacciones
    WHERE estado_transaccion = 'aprobada' -- Filtro de solo operaciones
    exitosas
)
SELECT
    id_transaccion,
    id_cliente,
    fecha_hora,
    monto_anterior,
    monto_usd AS monto_actual,
    -- Cálculo del factor multiplicador redondeado de forma limpia a dos
    decimales
    ROUND((monto_usd / NULLIF(monto_anterior, 0))::numeric, 2) AS
    factor_incremento
FROM transacciones_validas
WHERE monto_anterior IS NOT NULL
    AND monto_usd >= (5 * monto_anterior) -- Criterio: incremento igual o
    mayor a 5 veces
ORDER BY id_cliente, fecha_hora;
```

¿Qué hace el codigo?

1. **Uso de Expresiones de Tabla Comunes (WITH ... AS)**: Se define una subconsulta temporal nombrada `transacciones_validas`. Esto segmenta la lógica en dos pasos

(aislamiento de datos históricos y filtrado final), mejorando significativamente la legibilidad y el rendimiento de la consulta sobre el motor PostgreSQL.

2. **La Función de Ventana Analítica (LAG):** La instrucción clave es `LAG(monto_usd) OVER(PARTITION BY id_cliente ORDER BY fecha_hora)`.

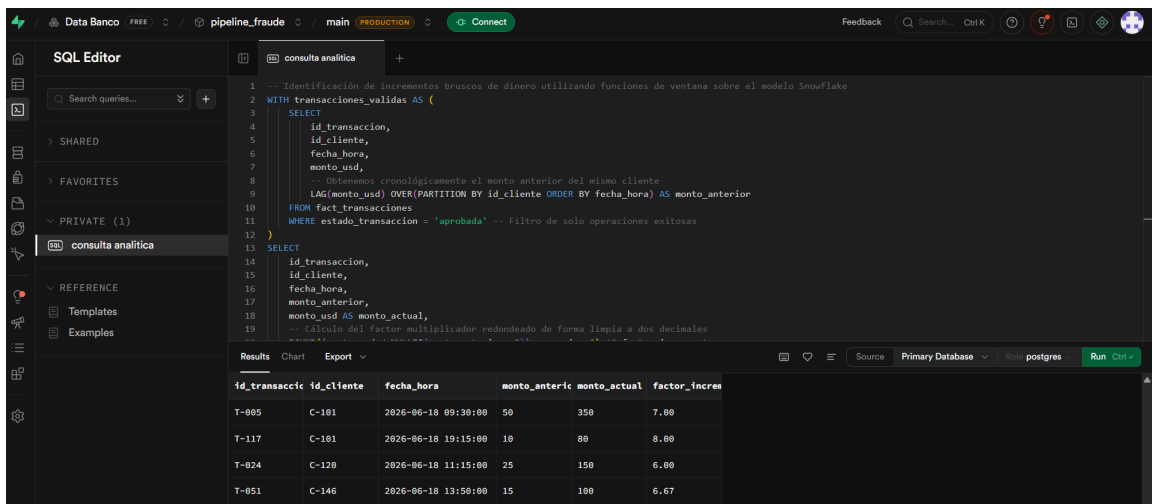
- **LAG(monto_usd)** : Permite acceder a una fila anterior dentro del mismo conjunto de datos sin necesidad de realizar costosos cruces de tablas (*Self-Joins*).
- **PARTITION BY id_cliente** : Agrupa el historial de transacciones de forma aislada para cada cliente, garantizando que el cálculo de un usuario no se mezcle con el de otro.
- **ORDER BY fecha_hora** : Organiza de manera estrictamente cronológica las operaciones, asegurando que el valor recuperado por `LAG` corresponda exactamente a la transacción inmediata anterior en el tiempo.

3. **Control de Errores Operativos (NULLIF):** En la fase de selección final se utiliza `NULLIF(monto_anterior, 0)`. Esta es una medida de seguridad en ingeniería de datos que transforma cualquier valor de "cero" en un valor nulo (`NULL`), evitando de forma matemática el error crítico de división por cero (*Division by Zero Error*) que detendría la ejecución del pipeline.

4. **Conversión y Redondeo Estricto (ROUND):** La operación `ROUND(...)::numeric, 2)` realiza una conversión explícita de datos (*casting*) hacia el tipo `numeric` requerida por PostgreSQL, permitiendo truncar y redondear el factor multiplicador a exactamente dos unidades decimales, garantizando un reporte analítico limpio (por ejemplo, transformando `6.66666...` en `6.66`).

5. **Evaluación del Umbral Financiero (WHERE):** La cláusula de salida filtra el universo de transacciones evaluando la condición de negocio: `monto_usd >= (5 * monto_anterior)`. Solo aquellos registros que hayan cumplido con un incremento de cinco o más veces respecto a su punto de partida histórico se consolidan como anomalías sospechosas.

Resultado desde Supabase



The screenshot displays the Supabase SQL Editor interface. The SQL Editor pane contains a query that identifies fraudulent transactions by comparing the current transaction amount with the previous transaction amount for the same client, filtered by approved status. The query uses window functions like LAG and ROUND. The Results pane shows the output of the query, displaying columns for transaction ID, client ID, transaction date, previous amount, current amount, and the calculated factor increment.

id_transacc	id_cliente	fecha_hora	monto_anterior	monto_actual	factor_incremento
T-005	C-101	2026-06-18 09:30:00	50	350	7.00
T-117	C-101	2026-06-18 19:15:00	10	80	8.00
T-024	C-120	2026-06-18 11:15:00	25	150	6.00
T-051	C-146	2026-06-18 13:50:00	15	100	6.67

Roles y Seguridad de los Datos (RLS)

Para garantizar que el pipeline cumpla con los estándares de seguridad de la industria, se implementó una estrategia de **Control de Accesos Basado en Roles (RBAC)** y **Seguridad a Nivel de Fila (RLS)** dentro de la base de datos en la nube (Supabase).

1. Arquitectura de Roles y Privilegios

El acceso a la base de datos está estrictamente desacoplado según el principio de *menor privilegio*:

- **Rol Administrador (`postgres`)**: Utilizado exclusivamente por el pipeline de Python (`script.py`) mediante conexión directa segura. Es el único rol con permisos para realizar operaciones DDL (crear/modificar tablas) e insertar registros de transacciones.
- **Rol Analítico**: Configurado en la interfaz de Apache Airflow (`postgres_conn_id='supabase_connection'`), aislando las credenciales del código fuente.
- **Rol Público (`anon`)**: Restringido por completo. No posee visibilidad de los datos de transacciones, clientes ni comercios.

2. Implementación de Row Level Security (RLS)

Para mitigar el riesgo de filtración o exposición accidental de datos en el esquema público de PostgREST, se habilitó **RLS** en todas las tablas resultantes del modelo Snowflake:

- `fact_transacciones`
- `dim_clientes`
- `dim_comercios`

Garantía de Producción: Al activar RLS, cualquier consulta externa o anónima es rechazada por defecto por la base de datos a menos que exista una política explícita. El script de automatización ETL no se ve afectado al operar bajo la identidad del Superusuario (`postgres`), garantizando la continuidad del flujo diario orquestado.

3. Políticas Aplicadas

- **Política de Aislamiento por Defecto (Default Deny)**: Al activar RLS en las tablas `fact_transacciones`, `dim_clientes` y `dim_comercios`, se habilitó la política restrictiva nativa de PostgreSQL. Esta regla deniega automáticamente cualquier petición HTTP/REST externa (API de Supabase), protegiendo la información sensible de transacciones.
- **Exclusión de Superusuario**: Se optó por no añadir políticas permisivas explícitas (como `FOR SELECT` o `FOR INSERT`), ya que el flujo de datos es estrictamente unidireccional: el pipeline orquestado pone los datos con privilegios de bypass (`postgres`), y no se

requiere que ninguna aplicación cliente interactúe con el modelo analítico Snowflake en esta etapa.

Entregable 3 (README .md)

1. Justificación de Calidad de Datos

Para que el equipo de analítica de fraude pueda trabajar correctamente y descubrir patrones sin sesgos operacionales, los datos deben estar limpios. Aplicar las reglas de limpieza en el script de Python es obligatorio por tres razones:

- **Evitar cálculos inflados (Manejo de Duplicados):** Si dejamos transacciones repetidas con el mismo ID, los reportes analíticos finales e indicadores de control mostrarán que se movilizó más dinero del real o que hubo más actividad de la verdadera, arruinando la precisión de los modelos de detección.
- **Evitar excepciones en el motor de ejecución (Manejo de Nulos):** Las herramientas analíticas y los frameworks matemáticos fallan o generan excepciones cuando encuentran valores nulos (NaN / NULL) en columnas de métricas monetarias. Al imputar un valor balanceado de 0.0 a las transacciones que fueron rechazadas y venían vacías, aseguramos la continuidad operativa de los análisis subsiguientes sin alterar el balance contable.
- **Optimización de consultas (Clasificación de Montos Inusuales):** Dejar precalculada la bandera lógica (es_monto_inusual) para transacciones internacionales mayores a \$1500 USD evita que los analistas o las consultas en producción tengan que realizar filtros pesados repetitivas veces en la base de datos; el dato ya viaja indexado y listo para su explotación.

2. Arquitectura, Flujo de Datos y Tecnologías

El camino que siguen los datos ha sido rediseñado para garantizar un acoplamiento modular y seguro entre la recolección, transformación y almacenamiento remoto en la nube.

Librerías Utilizadas:

- **Pandas (Manipulación y Modelado en Memoria):** Es el estándar de la industria en Python para el procesamiento de estructuras de datos bidimensionales (DataFrames). Se utiliza para la ingesta del CSV local de forma eficiente, aplicando vectorización para resolver la limpieza, imputación de nulos y la descomposición relacional en el esquema Snowflake.
- **Psycopg2-Binary (Conector y Driver Nativo de PostgreSQL):** Se seleccionó como la única interfaz de comunicación directa hacia la base de datos distribuida en Supabase. Al prescindir de ORMs pesados como SQLAlchemy, se eliminan capas intermedias de abstracción y potenciales conflictos de dependencias en el entorno virtual, permitiendo

una ejecución de scripts DDL y comandos `INSERT` puros a nivel de protocolo de red seguro (SSL).

Acceso al Código Fuente - Repositorio GitHub

Puede acceder y revisar el proyecto completo a través del siguiente enlace oficial:

Repositorio del Proyecto: [AmayaNicole / Associate-Data-Engineer](#)